Project 2: Cache & Memory Performance Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 16, 2026

Abstract

We profile the cache and memory hierarchy of an Intel i9-10900F with four purpose-built microbenchmarks, user-space core counters, system-wide memory-controller counters and Intel MLC. Zero-queue pointer-chase latency, taken on each level's plateau by a stated rule, is 1.80 ns in L1 (5.03 cycles), 5.21 ns on the L2 plateau with 4 KiB pages (this includes page-placement conflict misses; the L2 hit latency, measured with 2 MiB pages, is 4.33 ns), 18.6 ns in the 20 MiB L3 and 103 ns in DRAM with 4 KiB pages (83.4 ns with 2 MiB pages). MLC reports a lower 75.5 ns, because its random walk stays within a page. A loaded-latency probe running concurrently with up to nine streaming cores traces the throughput–latency curve directly. The knee is at four cores: 30.6 GiB/s, 95% of peak, at 1.37× the unloaded latency (MLC's own sweep: 30.2 GiB/s). More cores add queueing but no bandwidth, and latency more than doubles. With measured latency, Little's Law separates the 55.6 lines the controller serves from up to 152 that queue, and one core sustains at most 8.70 random demand misses. Counter-based AMAT fits explain the 37× random-SAXPY slowdown; verified 2 MiB pages remove a page-walk cost of up to 18.3 ns per access; and the memory-controller counters match the read-for-ownership traffic model within 1% at full load (4–5.5% below it with one core). We close with the cache-reuse bandwidth pitfall, where a 16 KiB buffer reports 32.5 GiB/s of nonexistent "DRAM" bandwidth.

1 Introduction

Every memory reference on this machine is served by one of four levels that differ by more than two orders of magnitude in latency: a 32 KiB per-core L1 data cache, a 256 KiB per-core L2, a 20 MiB shared L3, and DDR4 DRAM. Which level a kernel hits—and how many independent references it keeps in flight—decides its performance far more than its instruction count. This report measures that hierarchy on the Intel i9-10900F with purpose-built microbenchmarks, reads user-space hardware counters through `perf_event_open`, and interprets the numbers with AMAT and Little's Law.

This report answers:

1. What is the zero-queue access latency of each level, and how do latency and bandwidth change with access pattern, access size and read/write mix? (Sections 4 to 6)
2. Where does added concurrency stop buying throughput and start costing latency, and does Little's Law hold with measured latency? (Section 7)
3. How do working-set size, cache misses and TLB misses show up in counters and runtime? (Sections 8 to 10)
4. Which benchmark mistake produces misleading numbers? (Section 11)

Table 1: Measurement environment.

CPU	Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64)
SIMD ISA	sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm)
Topology	1 socket(s), 10 cores/socket, 1 threads/core (10 of 20 logical CPUs online); NUMA: 1 node(s)
SMT	off
Caches	L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance)
Memory	63 GiB total; DIMMs: not available (needs root/dmidecode)
Pages	base 4096 bytes; THP: always
OS / kernel	Arch Linux; 7.2.9-arch1-1
Frequency policy	governor: performance; driver: intel_pstate; turbo: disabled; EPP: performance
Pinning / isolation	single-thread runs and the loaded-latency probe pinned to CPU 2 (a physical core, not CPU 0); multi-threaded runs place one thread per physical core; no isolcpus
Compilers	gcc (GCC) 16.2.1 20260810; clang version 23.1.3
Tools	perf: perf version 7.2.11-1 (perf_event_paranoid 0); FIO: fio-3.42; MLC: present
GPU (optional)	NVIDIA GeForce RTX 3070; CUDA not available (no CUDA)
Env. snapshot	2026-10-21 20:46 EDT (each raw run directory also stores its own `env.txt`)

Summary of findings. Every number below is generated from the processed data (`data/summary.csv`, Section A). Zero-queue latency is 1.80 ns (5.03 cycles) in L1, 5.21 ns (14.6 cycles) on the L2 plateau with 4 KiB pages, which includes conflict misses (L2 hit latency with 2 MiB pages: 4.33 ns), 18.6 ns in L3 and 103 ns (289 cycles) in DRAM. Intel MLC agrees on L2 and measures lower L1, L3 and DRAM latency (75.5 ns), because its random walk stays within 4 KiB windows. Under load, bandwidth reaches 95% of its 32.1 GiB/s ceiling with only 1.37× the unloaded latency; beyond that, extra cores buy no bandwidth while latency doubles and nearly triples. MLC's loaded-latency sweep shows the same ceiling and knee. Little's Law with the measured loaded latency gives 72.4 lines in flight at the knee and up to 152 beyond it, of which the controller serves only 55.6. A single core keeps at most 8.70 random demand misses in flight—below its fill-buffer count—so several cores are needed to fill the pipe. Random SAXPY runs 37× slower per element once its footprint reaches DRAM, versus 6.1× for sequential order, and an AMAT model fitted to the counters attributes this to DRAM misses whose effective cost is 12.3 ns each—the zero-queue latency divided by 8.3-way memory-level parallelism. Memory-controller counters match the read-for-ownership model of writes within 1% at full load (4–5.5% below it with one core), and show that the adjacent-line prefetcher doubles DRAM traffic for sparse strided reads. Finally, a one-line change of working-set size turns a 10.7 GiB/s DRAM result into a fictitious 32.5 GiB/s "bandwidth" that is really L1 reuse (Section 11).

2 Methodology

2.1 Platform

All measurements were taken on the desktop system in Table 1: a ten-core Intel i9-10900F (Comet Lake, Skylake-family core) with per-core 32 KiB L1d and 256 KiB L2, a shared 20 MiB L3, and DDR4 memory on one NUMA node. The table is generated by `scripts/collect_env.sh` on the test host; each raw-data directory also stores its own snapshot (`env.txt`) taken immediately before that experiment, so the settings of every figure are on record even if the machine changes later.

2.2 Tools

All benchmarks share the harness in `ACS-Code/common` (monotonic raw timer, CPU pinning) plus a project header `src/memprof.h` (page-policy allocation, ring generators, counters), and raw per-repetition rows are reduced by `tools/summarize` (median and inter-quartile range). `scripts/derive.py` then computes every derived number quoted in the text (plateaus, knee, Little's Law, AMAT fit) into `summary.csv`; the report reads those values directly, so no number in the prose is copied by hand.

`mem_latency`	single-thread pointer-chase latency: working-set size, access size, node stride, page size, ring generator, timer, counters
`mem_bandwidth`	streaming bandwidth with T pinned threads: read/write/copy/triad/rwmix, access-size blocks, byte stride
`mem_loaded`	loaded latency: T load-generator cores (stream, rwmix, or k independent chases) plus a concurrent latency probe
`kernel_miss`	SAXPY loop whose footprint, order and page size vary, with counters
`clock_check`	diagnostic: fixed core clock vs. wall time lost by a pinned thread, idle DRAM traffic
Core counters	`perf_event_open(2)` called directly, per thread, user space only [3]
DRAM counters	uncore memory-controller events `data_reads`/`data_writes`, system-wide
Intel MLC	v3.13, run as a normal user (`run.sh mlc`) [1]

`perf` (perf version 7.2.11-1) is installed, but we want counts confined to the timed region of each repetition, so the benchmarks call `perf_event_open(2)` themselves, enable the counters immediately before each timed loop and read them immediately after it. Core events are counted for the measuring thread in user mode only (`exclude_kernel=1`): cycles, instructions, L1D read misses, LLC read references and misses, and dTLB read misses (completed page walks). They are opened independently so the kernel may multiplex them; counts are scaled by `time_enabled/time_running` and the smallest running fraction is recorded per row (`ctr_cover`). DRAM traffic is read from the client uncore memory-controller PMU (`uncore_imc`: `data_reads`, `data_writes`, one count per 64 B line, scaled by the PMU's `.scale` files), opened system-wide on CPU 0 around the same timed windows in `mem_loaded`, `mem_bandwidth` and `clock_check`. System-wide counting needs `perf_event_paranoid` ≤ 0, which the user set for these runs. Because the counts include all traffic in the machine, the background of an idle system is measured separately (`clock_check`) and quoted next to every result. Intel MLC v3.13 provides the external cross-check (Section 3). Without root it cannot switch off the hardware prefetchers, so it runs with `-e` (prefetchers left on) and `-r` (random access for the latency thread). MLC reports MB/s (10^6 B/s); we convert to GiB/s (2^{30} B/s) wherever we compare.

2.3 Controls

- **Pinning and isolation:** single-thread runs and the loaded-latency probe pinned to CPU 2 (a physical core, not CPU 0); multi-threaded runs place one thread per physical core. SMT: off. The machine is otherwise idle and experiments are run one at a time. All data in this report were collected on 2026-10-21, 20:22–20:39, in one `run.sh all` invocation (all experiments, MLC and `clock_check`).

- **Frequency:** governor performance, turbo disabled, so the core runs at its fixed 2.8 GHz base clock (equal to the TSC rate). Cycle figures are ns $\times$ 2.80 GHz. Two independent checks agree. User-mode cycles divided by wall time over the timed windows give a median of 2.77 GHz. The in-process add-chain probe logged by every run reads 2.78–2.79 GHz in all 1816 single-thread repetitions (2.69–2.79 GHz in the `mem_loaded` runs, where it runs just after nine generator

threads are started). The probe is still not used for conversion: in an earlier run it was disturbed by an ACPI event storm (Section 12).

- **Interference:** the ACPI general-purpose event that caused that storm (GPE 0x42) was masked for these runs; its count did not change during the experiments. In `clock_check`, a pinned thread kept 0.997–0.998 of the wall time.
- **Pages:** every buffer is a 2 MiB-aligned anonymous mapping whose page size is chosen by `madvise` *before first touch*: `MADV_HUGEPAGE` for "huge" and `MADV_NOHUGEPAGE` for "4k". The second is essential because THP is always: without it a "4 KiB" baseline can silently receive huge pages. Each run records the `AnonHugePages` actually granted (from `/proc/self/smaps_rollup`).
- **Pre-faulting and warm-up:** every buffer is written once to fault in its pages, then the ring or stream is traversed (or one untimed repetition is run) before timing, so neither page faults nor cold-cache effects fall inside the timed region.
- **Repetitions:** 8 per point; figures show the median with inter-quartile-range error bars.
- **Timing:** each timed interval is calibrated to $\geq$40 ms to 200 ms, far above the timer resolution, so timer overhead is negligible. The loaded-latency windows read generator progress counters at the start and end of the *same* interval in which the probe runs, so throughput and latency describe the same moment.
- **Code version:** the runs were started from a working tree on top of commit `29b8630` with uncommitted changes (`build.txt` records 37 changed paths). The measured sources, scripts, Makefile, shared harness and `tools/summarize` are byte-identical to those committed afterwards in `b00db66`. We verified this with `git diff b00db66`, which was empty for these paths before the later post-processing-only edits to `run.sh` and `derive.py`, and from the file times: every source was last modified before the binaries were rebuilt at 20:15, and the run started at 20:22. `run.sh` now warns loudly when the tree is dirty and records a hash of the uncommitted code diff in `build.txt`.
- **Provenance:** each `data/raw/<experiment>/<timestamp>/` directory holds the raw rows, `env.txt`, `build.txt` (git commit, compiler, flags, THP and counter settings), `commands.txt` (every command line) and `validate.txt` (the sanity gate of Section 3, run immediately before the experiment). Two directories are exceptions. `mlc/20261021-202254` runs Intel's binary rather than our build, so it has no `build.txt` or `validate.txt`; its raw data are MLC's text outputs plus `mlc_version.txt`. The earlier `clock_check/20261021-194648` run (kept as evidence for Section 12) has no `validate.txt`.

3 Validation and Sanity Checks

The microbenchmarks have no reference output to diff, so `scripts/validate.sh` (also `make test`) gates them against physical invariants that must hold on any sane hierarchy. `run.sh` executes it immediately before every experiment and stores the full output as `validate.txt` in that experiment's raw directory, so each dataset carries its own evidence that the checks passed on the machine state it was measured in. The checks are:

- **Monotone levels.** Random pointer-chase latency grows strictly across levels: L1 < L2 < L3 < DRAM (measured values in Table 2).

- **Prefetcher check.** Sequential DRAM "latency" is more than 2× below random DRAM latency, confirming the stream prefetcher is engaged for sequential access and defeated by the randomized chase.
- **Bandwidth ceiling.** Single-core L1-resident read bandwidth exceeds the DRAM-resident rate, and ten threads exceed one thread.
- **Loaded > unloaded.** The `mem_loaded` probe is slower under four streaming cores than on an idle memory system—the minimum requirement for a loaded-latency instrument.
- **Frequency plausibility.** The in-process add-chain probe lands in 1 GHz to 6 GHz. This is only a coarse check: cycle conversions use the fixed 2.8 GHz clock, which the cycle counter confirms (Section 2).
- **Instrumentation (reported, not gating).** User-space counters open and return values; a 2 MiB-page request is actually granted (`AnonHugePages` > 0) and the 4 KiB policy yields none.

External cross-check: Intel MLC. `run.sh mlc` runs Intel MLC v3.13 with the latency thread on CPU 2 and load threads on the other nine cores, using `-e -r`: without root MLC cannot switch off the prefetchers, so its latency thread uses random access. Table 2 compares its idle latency with ours level by level, and Section 7 compares its loaded-latency sweep with our knee. Agreement is close where the methods measure the same thing; the differences are explained by how the two tools walk memory (Section 4).

Additional internal checks. Every zero-queue size is measured twice: the primary ring is built with a 32-bit LCG and timed with `CLOCK_MONOTONIC_RAW`; a second ring is built with xorshift64* (a different permutation, seed and range reduction) and timed with `rdtscp`. On the DRAM plateau they differ by 0.11%, and `validate.sh` fails if they differ by more than 15%. The $T = 0$ point of `mem_loaded` measures unloaded DRAM latency with a separately written, pointer-based ring in a different binary: 103 ns, against 103 ns from `mem_latency` at the same 256 MiB size.

An L1 latency of 5.03 cycles also matches the published 4–5 cycle load-to-use latency of this core, anchoring the ns→cycle conversion.

Timer and loop overhead are bounded by calibrating every timed interval to tens of milliseconds (Section 2); the chase loop body is a single dependent load, so no overhead subtraction is needed.

4 Zero-Queue Baselines

Hypothesis (from first principles, then compared with the data)

With a single dependent access in flight, latency should rise in steps at the L1 (32 KiB), L2 (256 KiB) and L3 (20 MiB) capacities. We expect roughly 4–5 cycles for L1, ∼12 cycles for L2, a few tens of cycles for L3, and ∼80 ns (hundreds of cycles) for DRAM, matching Skylake-family numbers.

Fixed	one thread pinned to CPU 2; random (Sattolo) pointer chase; one node per 64 B line
Varied	working-set size inside each level's plateau window; page size (DRAM only); method (primary vs. cross-check)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh zero_queue`

Table 2: Zero-queue random-access latency per level (median over the plateau sizes; each size is the median of 8 repetitions); cycles = ns × 2.8 GHz, the fixed core clock with turbo disabled (Section 2). "min–max" is the spread across plateau sizes, and "IQR%" the largest inter-quartile range of the 8 repetitions at any one of those sizes, relative to its median. "x-check" is our second method (xorshift64* ring, `rdtscp` timing), with Δ_x its signed difference from the primary value. "2 MiB" is the same chase with 2 MiB pages: for DRAM it removes the page-walk component; for L2 (taken from the working-set sweep at the same plateau sizes, Section 8) it removes the 4 KiB-page conflict misses and is the L2 hit latency. "MLC" is Intel MLC `--idle_latency -e -r` at 16 KiB, 128 KiB, 4 MiB and 256 MiB (median of 3 runs), with Δ_{MLC} its signed difference from our value (negative: MLC is lower). Generated from `data/zero_queue_table.csv`.

Level	Plateau sizes	ns	cycles	min	max	IQR%	x-check	Δ_x %	2 MiB	MLC	Δ_{MLC} %
L1	8–24 KiB	1.80	5.03	1.79	1.80	0.19	1.80	0.0	—	1.40	−22.0
L2	64–192 KiB	5.21	14.6	4.31	6.65	0.21	4.94	−5.3	4.33	5.40	3.6
L3	0.5–15 MiB	18.6	52.1	14.4	24.7	10	18.6	−0.2	—	15.8	−15.1
DRAM	≥80 MiB	103	289	98.8	105	0.27	103	0.1	83.4	75.5	−26.8

The ring is a single random cycle over one node per cache line, so each load depends on the previous one and exactly one request is outstanding—zero-queue latency. Cycles are converted with the fixed 2.8 GHz core clock, which the cycle counter confirms (Section 2).

Plateau rule. A level's latency is *not* read at its capacity edge, where the set is partly spilling (e.g. 256 KiB is a transition point, not an L2 point). Instead, `derive.py` takes the median, over all measured sizes s with $2\,C_{prev} \le s \le 0.75\,C_{level}$, of the per-size medians ($C_{prev} = 4$ KiB for L1); DRAM uses every size $\ge 4\times$ the L3. The same values feed the abstract, introduction and conclusion through generated macros, so they cannot disagree.

4.1 Analysis

Latency is flat at 1.80 ns (5.03 cycles) while the set fits in L1. The L2 plateau with 4 KiB pages is 5.21 ns (14.6 cycles), the median of sizes that measure 4.31–6.65 ns. This headline value is *not* the L2 hit latency: the high end comes from 128 KiB and 192 KiB, where 4 KiB pages already cause L2 conflict misses (Section 8). With 2 MiB pages the same plateau sizes give 4.33 ns (about 12 cycles), as does the low end of the 4 KiB range; that is the L2 hit latency. Latency spans 14.4–24.7 ns across the L3 (median 18.6 ns); L3 latency is not a single number, because larger sets also miss the second-level TLB. It reaches 103 ns (289 cycles) in DRAM. With 2 MiB pages the DRAM figure drops to 83.4 ns; the 19.6 ns difference is the page walk that every random 4 KiB access beyond the TLB reach pays (Section 10). Our second method agrees within 5.3% at every level: the largest difference is on the L2 plateau, where it reads 5.3% lower, and on DRAM it differs by 0.11%. Against the hypothesis, L1 (5 cycles), L2 (12 cycles without conflicts) and L3 match. DRAM with 4 KiB pages is slower than the ∼80 ns we expected, and the 2 MiB-page value shows that the excess is mostly the page walk.

Comparison with Intel MLC. MLC agrees with our 4 KiB-page L2 plateau (5.40 ns, 3.6% relative to ours) but is lower at the other levels: L1 1.40 ns (−22%), L3 15.8 ns (−15%), and DRAM 75.5 ns (−27%). The differences follow from how the tools walk memory, not from noise:

- **L1.** MLC's 4 clocks versus our 5 cycles fit the known 4-cycle load-to-use latency for a plain pointer dereference. Our index ring forms `ring[p]` with a scaled-index address, which costs one extra cycle.
- **DRAM.** MLC returns the same 75.5 ns with and without `-L` (2 MiB pages). So its random walk takes no page walks, consistent with its `-D` random-access range of 4 KiB (the default): consecutive accesses stay inside one page. Our ring is random over the whole buffer, so even with 2 MiB pages MLC's figure is 9.5% below ours (83.4 ns), and with 4 KiB pages our ring also pays the walk. Accesses that stay close together should also hit open DRAM rows more often.

MLC therefore measures a best-case local latency; our numbers are the cost of truly random access. Both are valid, and we keep ours as the zero-queue baseline because the rest of the report (AMAT, TLB) needs fully random access.

Caveats: the L3 and DRAM figures are a single-core view of shared resources on an otherwise idle system. Sequential "latency" (Section 5) is prefetcher throughput, which is why the chase is randomized.

5 Access Pattern and Granularity

Hypothesis (from first principles, then compared with the data)

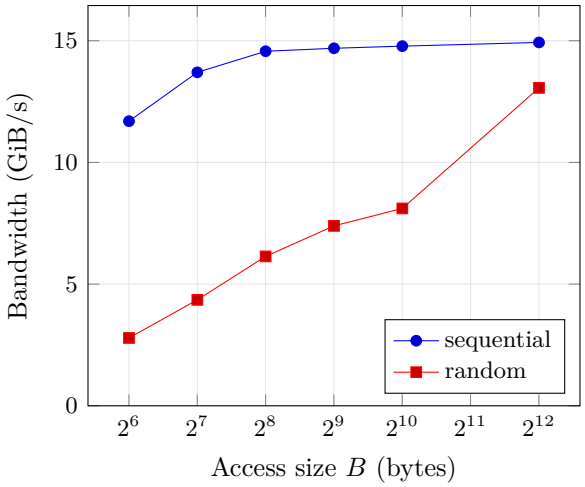
Access size. Reading a whole B-byte block per access should leave sequential bandwidth roughly unchanged (the stream is the same) but raise random bandwidth steeply with B, because a larger block amortises one DRAM latency over more lines and lets the adjacent-line and stream prefetchers help. The latency of one dependent B-byte access should grow far less than B, since its lines are fetched in parallel. *Stride.* With one 8 B element used per stride, useful bandwidth should fall roughly as 1/stride once the stride reaches a 64 B line.

Fixed	1 thread pinned to CPU 2; 256 MiB DRAM-resident region; 100% reads; 4 KiB pages
Varied	pattern (sequential, random); access size $B = 64 \ldots 4096$ B (primary); byte stride 8–1024 (secondary)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh pattern`

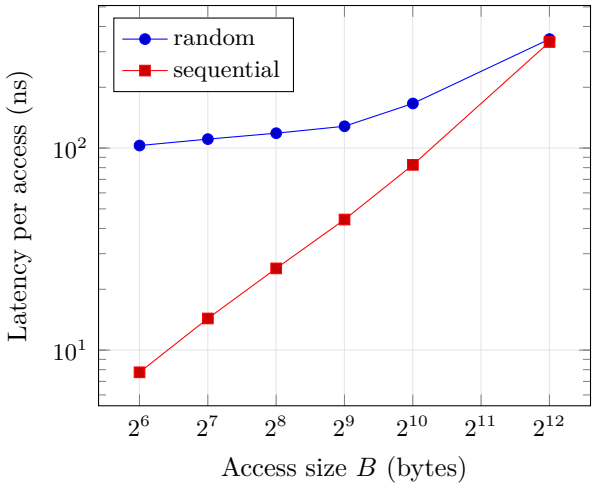
Definitions. *Access size B*: the buffer is cut into B-byte blocks and every byte of a block is read (256 bit vector loads) before the next block; sequential visits blocks in address order, random in a shuffled order. For *latency*, `mem_latency -access B` chains the blocks: the next block's address depends on all lines of the current block, so one access is one dependent B-byte request. *Stride* (secondary, Figure 2) keeps the earlier definition: one 8 B element used per stride, scalar FP loads.

5.1 Analysis: access size

At $B = 64$ B sequential blocks stream at 11.7 GiB/s and random blocks at 2.79 GiB/s (4.19× apart). Growing the block to 256 B and 1024 B lifts random bandwidth to 6.14 GiB/s and 8.11 GiB/s (sequential: 14.6 GiB/s, 14.8 GiB/s), shrinking the gap to 1.82×; at 4096 B random blocks reach 13.1 GiB/s, close to the sequential 14.9 GiB/s. Within a block lines are contiguous, so their misses overlap and the prefetchers engage. Sequential bandwidth is flat from 256 B up, as expected; it is lower only at 64 B, where the per-block loop overhead is paid on every line. The latency panel tells the same story per request. A random dependent 64 B access costs 103 ns, but a 1024 B one costs



(a) Streaming bandwidth of B-byte blocks.



(b) Latency of one dependent B-byte access.

Figure 1: Access pattern and access size, DRAM-resident, one core. Strided rows of `pattern_bw.csv` ($B = 0$) are dropped from the log axis and shown in Figure 2.

only 166 ns—16 lines for far less than 16 latencies—so a single dependent chain moves 5.74 GiB/s instead of 0.579 GiB/s. Sequentially chained blocks cost 7.77 ns (64 B) and 82.6 ns (1024 B); the difference from the random values is the part of the DRAM latency the stream prefetcher hides by fetching the next block early, so sequential "latency" is partly throughput. The hiding fades as blocks grow: at 4096 B a sequential access costs 335 ns, no better than random (346 ns). Every block then starts a new 4 KiB page, and the stream prefetcher does not prefetch across page boundaries, so each dependent access begins with a full miss.

5.2 Analysis: stride, and the line-traffic anomaly

Sequential useful bandwidth is 10.8 GiB/s at unit (8 B) stride and 1.96 GiB/s at 64 B. At 64 B stride every access still pulls a whole line, so the *line* traffic is 8× the useful bandwidth: 15.7 GiB/s, i.e. 1.45× the unit-stride rate. A unit-stride scalar read is therefore *not* a measure of the single-core streaming limit, although it is the obvious kernel to use for one. The explanation is on the core side, not in memory: the unit-stride scalar kernel spends eight loads and eight FP adds on every line, so fewer distinct lines fit in the out-of-order window and fewer misses are in flight. Kernels that spend fewer instructions per line move more lines: one load per line (64 B stride) gives 15.7 GiB/s, and full-line vector reads of large blocks give up to 14.9 GiB/s. The vector kernel at 64 B blocks, which still pays per-block loop overhead, is only 1.08× the scalar kernel (11.7 GiB/s). The single-core streaming limit is therefore about 15 GiB/s to 16 GiB/s, and no measured line traffic exceeds it. The loaded-latency study (Section 7) uses a two-loads-per-line vector stream. Contrary to our hypothesis, useful bandwidth does *not* keep falling as 1/stride beyond one line: it is 0.912 GiB/s at 256 B and 0.799 GiB/s at 1024 B, so touched-line traffic drops from 15.7 GiB/s to 7.30 GiB/s and 6.39 GiB/s. Once each access is a separate line, the kernel is limited by how many misses it keeps in flight, not by bandwidth; skipping lines saves bandwidth it was not using. The memory-controller counters show where the bytes go. At 64 B stride DRAM reads are 15.1 GiB/s, 0.961× the touched-line traffic. At 256 B and 1024 B strides DRAM reads are 2.03× and 2.04× the touched lines (14.8 GiB/s and 13.0 GiB/s): the spatial (adjacent-line) prefetcher fetches the other half of each 128 B pair, so every useful 8 B costs two lines of DRAM traffic. Random 64 B blocks show the same pairing: DRAM reads

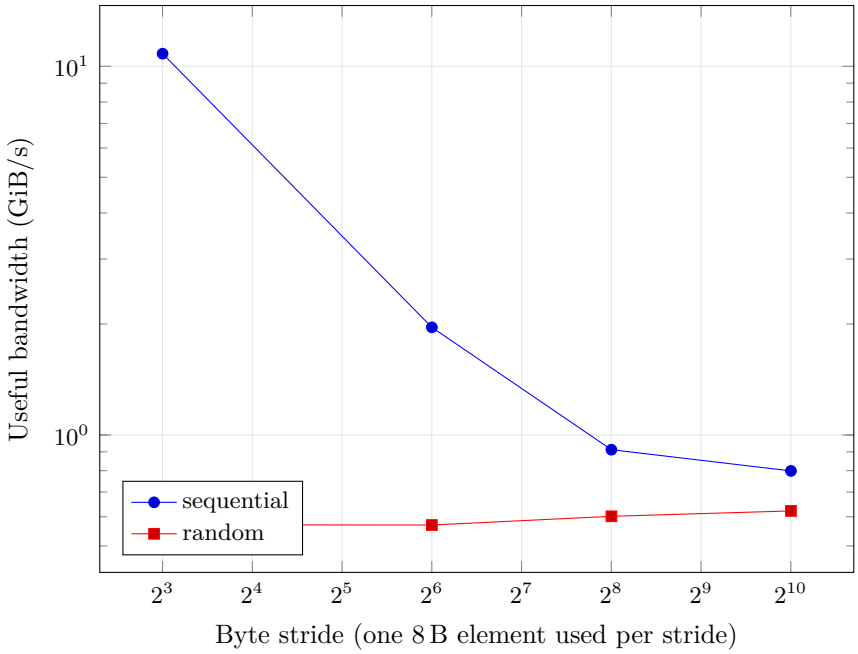


Figure 2: Secondary result: useful bandwidth vs. byte stride with scalar 8 B loads.

are 5.49 GiB/s for 2.79 GiB/s of useful blocks. Random strided access is flat at 0.571–0.622 GiB/s of useful data: each access is an independent miss, so stride barely matters.

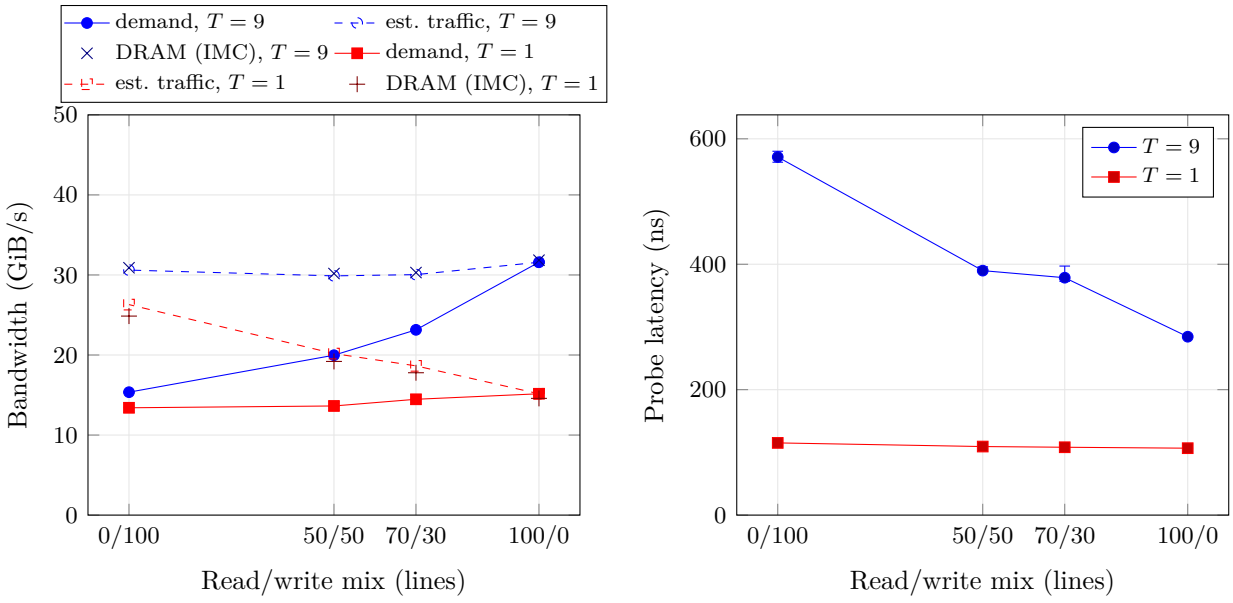
6 Read/Write Mix

Hypothesis (from first principles, then compared with the data)

Pure reads should reach the highest *demand* bandwidth. Every written line first needs a read-for-ownership (RFO) and later a write-back, so a written line costs about two lines of DRAM traffic and adds read/write bus turnarounds. Demand bandwidth should therefore fall as the write fraction grows, while estimated DRAM traffic stays closer to constant; and loaded latency should be highest for the write-heavy mixes, whose write-backs compete with the probe's reads.

Fixed	128 MiB per generator core; sequential; ordinary (write-allocate) full-line stores; 4 KiB pages
Varied	reads per 10 *lines*: 10 (100/0), 7 (70/30), 5 (50/50), 0 (0/100); $T = 1$ and 9 generator cores
Latency	concurrent random pointer-chase probe on CPU 2 (`mem_loaded`); $T = 0$ gives the unloaded value
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh rw_mix`

Mix definition. Of every 10 consecutive 64 B lines, the first R are read and the remaining $10 - R$ are fully overwritten. Mixing at *line* granularity makes the ratio hold for the traffic the memory system sees. Mixing individual 8 B elements instead is a methodological trap: with 7 reads and 3 writes per 10 elements, every line of the source is read *and* every line of the destination written, so a nominal "70/30" moves about three lines per useful line and measures the access pattern, not the mix (Section 13). "Demand" bandwidth counts the bytes the program reads or writes; "est. traffic" adds one RFO line per written line (written lines count twice); "DRAM (IMC)" is the read plus write traffic measured by the memory-controller counters over the same window.



(a) Throughput.

(b) Loaded latency of a random read.

Figure 3: Read/write mixes: throughput and concurrently measured latency. Demand bandwidth is what the program reads and writes; "est. traffic" adds one RFO line per written line; "DRAM (IMC)" is the memory-controller read+write count over the same windows. The unloaded probe latency is 103 ns.

6.1 Analysis

With nine generator cores, demand bandwidth is 31.6 GiB/s for 100% reads, 23.1 GiB/s at 70/30, 20.0 GiB/s at 50/50 and 15.4 GiB/s for 100% writes, falling with the write fraction as predicted. Counting one RFO per written line, the estimated DRAM traffic is 31.6 GiB/s, 30.0 GiB/s, 29.9 GiB/s and 30.6 GiB/s. The memory-controller counters measure 31.8 GiB/s, 30.3 GiB/s, 30.2 GiB/s and 30.9 GiB/s (within about 1% of the estimate at every mix), against an idle background of 0.088 GiB/s. At full load the RFO model therefore holds to within 1%: for pure writes the controller reads 15.6 GiB/s (the RFOs) and writes 15.3 GiB/s. Total DRAM traffic stays at the controller's ceiling of about 30 GiB/s whatever the mix; MLC's peak-injection bandwidth agrees (31.8 GiB/s all reads, 30.0 GiB/s at 3:1, 30.6 GiB/s at 1:1; MLC reports 34 102 MB/s for all reads, 1 MB = 10^6 B). The cost of writes is therefore the RFO, not a loss of controller efficiency.

The concurrently measured probe latency is 284 ns, 379 ns, 390 ns and 571 ns for the four mixes (unloaded: 103 ns). At the same total DRAM traffic, only pure writes double the read latency relative to pure reads (2.01×); the 70/30 and 50/50 mixes raise it by about a third (1.33× and 1.37×). Write-backs and RFOs occupy the same channels and queues as the probe's reads, and read/write bus turnarounds lengthen every read's wait.

A single generator core ($T = 1$) is not controller-bound: demand bandwidth falls only slightly from 15.2 GiB/s (reads) to 13.4 GiB/s (writes), while measured DRAM traffic grows from 14.6 GiB/s to 24.9 GiB/s. With one core the measurement is 0.945–0.962× the RFO-counted estimate, for pure reads as well as for mixes; with nine cores the gap vanishes. We did not resolve this gap, so it is a limitation of the $T = 1$ traffic comparison. *Untested hypothesis:* it is mostly the estimate's own error, because the throughput counts lines the generator touches that are still in the 20 MiB L3 from the previous pass (one core's 128 MiB stream may not flush the cache completely in every window).

A test would repeat $T = 1$ with a buffer many times the L3 and check whether the ratio moves to 1. A second, also untested, hypothesis explains why demand bandwidth falls so little with writes at $T = 1$: stores retire without waiting for their RFO, so a core can keep more lines moving when part of its stream is stores. The probe barely notices one core: 107–115 ns across the four mixes. As named-kernel references on all ten cores, `copy` delivers 20.0 GiB/s and `triad` 21.9 GiB/s of demand bandwidth. The counters show their RFOs too: copy reads 20.1 GiB/s and writes 10.0 GiB/s (2:1, source plus RFO), and triad reads 22.1 GiB/s and writes 7.33 GiB/s (3:1). Non-temporal stores, which skip the RFO, were out of scope for the required mixes.

7 Intensity Sweep and the Throughput–Latency Knee

Hypothesis (from first principles, then compared with the data)

A single core cannot saturate the DRAM controller because it can keep only about a dozen demand misses in flight (its L1 fill buffers). Adding load should raise delivered bandwidth until the memory controller saturates in the low tens of GiB/s (assuming the usual two DDR4 channels of this desktop platform; DIMM population was not recorded); past that point extra load should buy almost no bandwidth but sharply raise the latency of every request (queueing). By Little's Law, saturation needs tens of outstanding lines—more than one core provides.

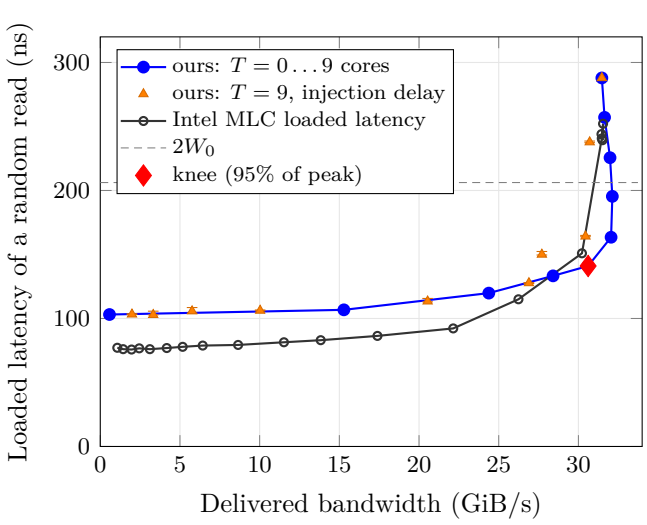
Generators	`mem_loaded`: $T = 0 \ldots 9$ cores (all physical cores except CPU 2), each streaming its own 128 MiB region with 256 bit loads; then $T = 9$ throttled by an injection delay of 8–2048 empty iterations per line
Probe	random pointer chase over 256 MiB on CPU 2, measured in the same window as the generators' throughput
Known MLP	T cores each running k independent random chases: $T = 1$, $k = 1 \ldots 32$; $T = 2 \ldots 9$, $k = 8$
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh intensity`

Method. Each point reports *both* axes from one measurement window: generator threads publish progress counters, the main thread snapshots them, runs the latency probe for 200 ms, and snapshots again. Throughput is all lines moved (generators plus probe) per second; latency is the probe's time per dependent access—a demand read issued *under* that load, i.e. MLC's "loaded latency". $T = 0$ is the unloaded latency of the same setup ($W_0 = 103$ ns).

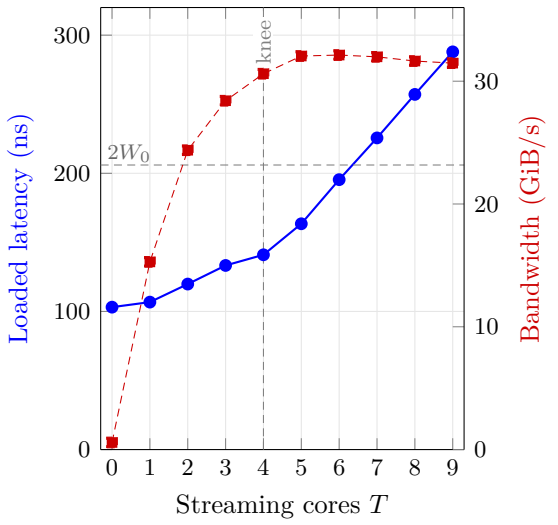
Knee criterion. We define the knee as the first point, in order of *offered* load ($T = 0, 1, \ldots, 9$), at which delivered bandwidth reaches 95% of the peak. Past it, extra load can buy at most 5% more bandwidth. The point of maximum throughput/latency ("power") is a second criterion. As a latency reference we also report where loaded latency first reaches $2W_0$, interpolated between the two bracketing points; on this machine that happens only once bandwidth has already peaked, so it marks deep saturation rather than the knee. All three come from the CSV (`derive.py`), not from the eye.

7.1 Knee and diminishing returns

The curve has the textbook shape (Figure 4, Table 3). The knee is at $T = 4$: bandwidth has climbed from the probe's own trickle to 30.6 GiB/s, 95% of the 32.1 GiB/s peak, while latency has risen only from 103 ns to 141 ns (1.37×). The throughput/latency maximum falls on the same point ($T = 4$, 30.6 GiB/s, 141 ns), so both criteria agree. Every core added after the knee buys almost no bandwidth: the peak is reached at $T = 6$, and $T = 9$ still delivers 31.5 GiB/s. Latency, however,



(a) Latency vs. delivered bandwidth.



(b) Latency (left, blue) and bandwidth (right, red) vs. cores.

Figure 4: Measured throughput–latency curve. (a) Our load levels with Intel MLC's loaded-latency sweep for comparison (MLC's MB/s converted to GiB/s). (b) The thread sweep against offered load: bandwidth flattens at the knee while latency keeps rising. Each of our points is one load level; both coordinates are medians of 8 windows (bars: IQR of latency). The knee is the first load level that reaches 95% of peak bandwidth; the dashed horizontal line marks twice the unloaded latency.

climbs steadily to 288 ns. It crosses $2W_0$ only at 32.1 GiB/s, between $T = 6$ and $T = 7$, i.e. at the bandwidth peak itself. (In the injection-delay sweep at $T = 9$ the crossing is at 30.6 GiB/s.) The memory-controller counters confirm the throughput axis: at $T = 4$ and $T = 9$ measured DRAM traffic is 1.01× and 1.01× the delivered bandwidth. Beyond the knee the memory controller, not the cores, is the bottleneck: requests no longer finish faster, they queue. *Theoretical DRAM peak: not stated.* The DIMM speed and channel population were not recorded (`env.txt` reports "DIMM speed: not available"; reading it needs root and `dmidecode`), so we do not express the knee as a fraction of the theoretical peak; the measured ceiling above is the reference.

Comparison with Intel MLC. MLC's `--loaded_latency` sweep (nine load cores, injection delay from 20 000 down to 0 cycles, latency thread on CPU 2) is plotted in Figure 4. It starts lower, at 77.1 ns, for the access-pattern reasons discussed in Section 4. By the same 95% rule, MLC's knee is at 30.2 GiB/s and 151 ns (peak 31.5 GiB/s, injection delay 50), against our 30.6 GiB/s. MLC's own latency doubles at 30.3 GiB/s. At full load it reports 244 ns. The two instruments agree on the ceiling (within 2%) and on the shape: latency stays near its idle value until the controller approaches 30 GiB/s to 32 GiB/s and then rises about threefold with almost no further bandwidth.

7.2 Little's Law with measured latency

For every load level we compute the mean number of lines in flight, $L = \lambda W$ [2], with λ the delivered line rate and W the *measured* loaded latency of that same window. At the knee ($T = 4$), $L = 72.4$ lines. For comparison, $L_0 = \lambda W_0$ uses the unloaded latency, the usual shortcut when loaded latency is not measured. L_0 rises to 55.6 lines at saturation and then stays flat: that is the concurrency the memory system can actually *serve* at about 30 GiB/s. L, in contrast, keeps growing to 152 lines

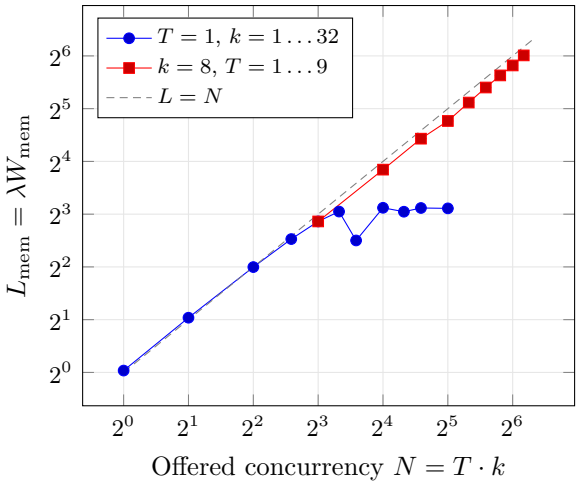
Table 3: Little's Law across the thread sweep (no injection delay), generated from `data/littles_law.csv`. λ = delivered lines/s (64 B lines), W = loaded probe latency, $L = \lambda W$; $L_0 = \lambda W_0$ with the unloaded W_0.

T	BW (GiB/s)	λ (Mlines/s)	W (ns)	$L = \lambda W$	$L_0 = \lambda W_0$
0	0.58	10	103.1	1.0	1.0
1	15.29	256	106.7	27.4	26.4
2	24.38	409	119.8	49.0	42.2
3	28.41	477	133.3	63.5	49.1
4	30.61	514	141.0	72.4	52.9
5	32.05	538	163.5	87.9	55.4
6	32.13	539	195.4	105.3	55.6
7	31.98	537	225.6	121.1	55.3
8	31.64	531	257.1	136.5	54.7
9	31.47	528	287.9	152.0	54.4

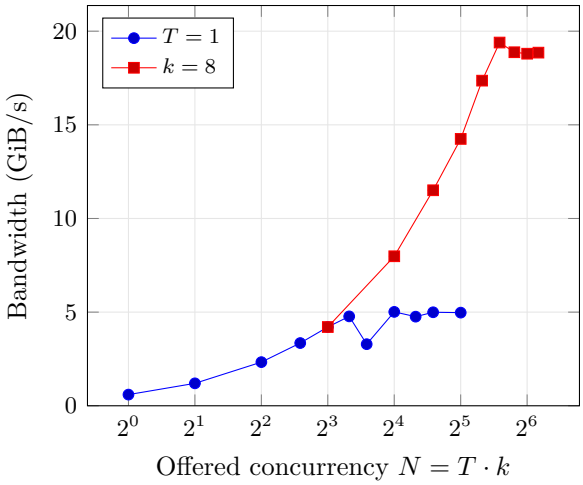
at $T = 9$; the difference $L - L_0$ is requests waiting in queues, which is exactly why latency rises without any bandwidth gain.

What the known-concurrency sweep shows. Computing $L = \lambda W$ with W taken as each chain's own step time gives L/N between 1.00 and 1.01. That is *not* a validation of Little's Law: with that W, $\lambda W = N$ holds by construction, so it only confirms the progress accounting. The informative quantity is $L_{\text{mem}} = \lambda W_{\text{mem}}$, where W_{mem} is the latency of a memory request measured at the same time by the probe (103 ns on average with one generator core). One core reaches its best random-read bandwidth, 5.01 GiB/s, at $k = 16$, and its L_{mem} never exceeds 8.70 lines (Figure 5). Chains beyond that do not reach memory any sooner: they wait inside the core, so the per-chain step time grows instead (to 384 ns at $k = 32$). About nine outstanding demand misses per core is consistent with the 12 L1D fill buffers of this Skylake-family core. One point, $k = 12$, is an outlier (slower than both $k = 10$ and $k = 16$, with the same memory latency); we did not resolve this generator artifact and use the envelope. With $k = 8$ per core, adding cores raises bandwidth to 19.4 GiB/s at $T = 6$ ($L_{\text{mem}} = 42.2$, $W_{\text{mem}} = 130$ ns). Random demand reads thus reach only about 60% of the streaming peak: without prefetching, nine cores of this kind keep about seven lines each in flight and the memory system queues long before it is fully used.

Reconciling one core's streaming concurrency with the fill buffers. The streaming sweep gives one core $L = 27.4$ lines in flight at 15.3 GiB/s (Table 3), about three times the 8.70 demand misses one core sustains. There is no contradiction: in a sequential stream most lines are brought in by the L2 hardware prefetcher, whose requests are tracked in the L2's own request queue rather than in the L1 fill buffers, so a streaming core keeps more lines in flight than a core issuing random demand misses. Quoting the fill-buffer count as the per-core limit is correct only for demand misses; the two measurements quantify both limits. The 55.6 lines the controller serves at saturation are about two streaming cores' worth of concurrency (with queueing, four cores are needed to reach 95% of peak), but six random-access cores' worth.



(a) Lines in flight in memory vs. offered N.



(b) Random-read bandwidth vs. N.

Figure 5: Known-concurrency sweep: k independent random chases per core (median of 8 windows, bars: IQR). L_{mem} uses the memory latency W_{mem} measured by the concurrent probe; the dashed line is $L = N$, which would be reached only if every chain always had a request in the memory system.

8 Working-Set Sweep

Hypothesis (from first principles, then compared with the data)

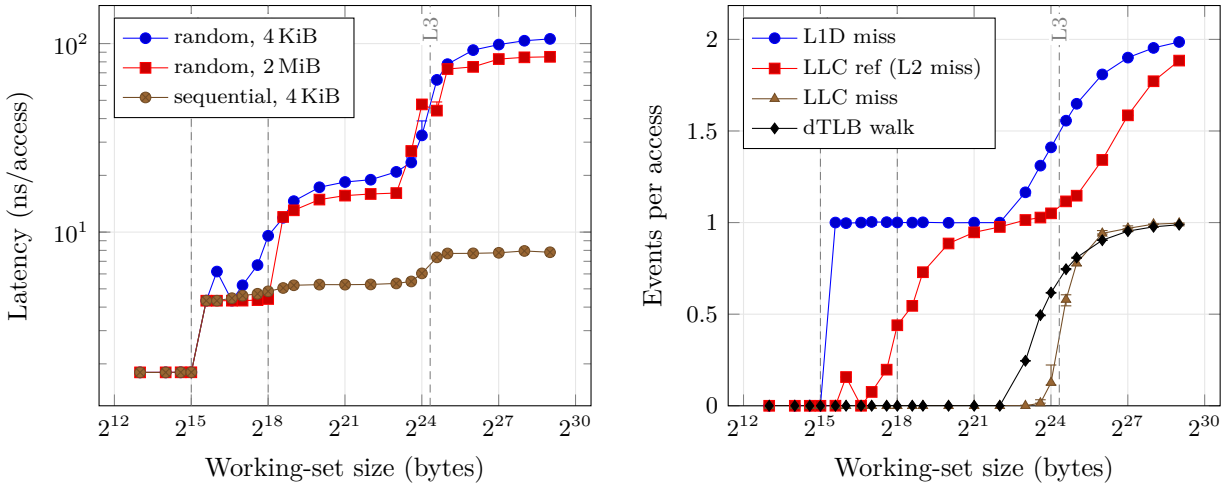
Pointer-chase latency should trace the capacity staircase: flat while the set fits in L1 (32 KiB), a step up through the 256 KiB L2, a wider plateau across the 20 MiB L3, and a final jump to DRAM latency. The counters should show the cause: L1D misses per access rising from 0 to 1 past 32 KiB, LLC misses per access rising to 1 past 20 MiB, and dTLB walks appearing once the set exceeds the 4 KiB-page TLB reach (∼6 MiB) but not with 2 MiB pages.

Fixed	1 thread pinned to CPU 2; one node per 64 B line; user-space counters
Varied	working-set size 8 KiB–512 MiB; random vs. sequential; 4 KiB vs. 2 MiB pages (random)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh working_set`

8.1 Analysis

The latency staircase and the counters line up level by level (Figure 6). L1D misses per access jump from ≈0 to 1 between 32 KiB and 48 KiB, exactly at the L1 capacity. With 4 KiB pages, LLC references (L2 misses) already appear inside the L2 window, well before the 256 KiB capacity, but not monotonically: 0.16 per access at 64 KiB, 0.00017 (essentially none) at 96 KiB, 0.076 at 128 KiB and 0.20 at 192 KiB; they approach one per access only by 1 MiB to 2 MiB. Page walks start at 8 MiB, just past the ≈6 MiB STLB reach, and explain why the 4 KiB latency already climbs inside the L3 region (20.9 ns vs. 16.1 ns with 2 MiB pages at 8 MiB). LLC misses appear from 12 MiB to 16 MiB, before the nominal 20 MiB, and reach one per access by 64 MiB.

A page-size crossover at the L3 edge. Near the L3 capacity the two page sizes swap order. At 12 MiB and 16 MiB the 2 MiB-page chase is *slower* than the 4 KiB-page chase (26.9 ns vs. 23.4 ns,



(a) Latency.

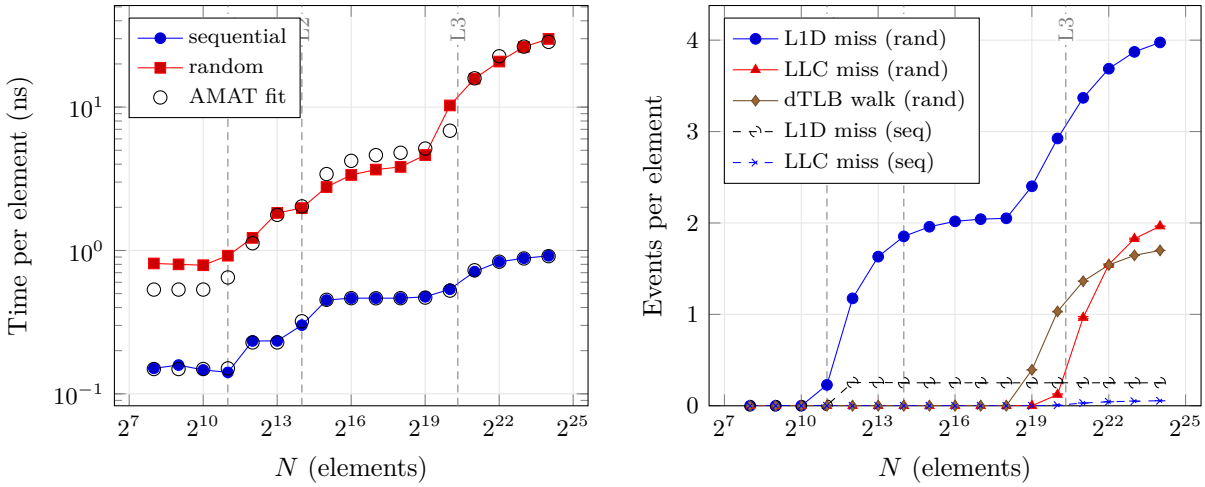
(b) Counters, random chase, 4 KiB pages.

Figure 6: Working-set sweep. Left: latency with cache capacities marked. Right: user-space counters per access for the random 4 KiB-page chase; each step in latency coincides with a miss ratio rising towards one.

and 47.6 ns vs. 32.6 ns), with more LLC misses per access (0.15 vs. 0.015, and 0.46 vs. 0.13) and no page walks at all. From 24 MiB on, the order reverts (44.1 ns vs. 64.2 ns), and the 2 MiB curve is not even monotonic (16 MiB is slower than 24 MiB). Page-table lines competing for the L3 therefore cannot explain the early LLC misses: the configuration with the most early misses has no page tables to walk. *Untested hypothesis:* the cause is physical placement in the L3. The L3 is split into slices selected by a hash of the physical address, and sets are indexed by lower address bits. A 12 MiB to 16 MiB buffer of 2 MiB pages occupies only 6–8 physically contiguous frames, so any unevenness in how those few frames map to slices and sets is shared by every line in them; 4 KiB pages scatter the buffer over thousands of random frames (page colours), which averages the occupancy out. A test would repeat the 12 MiB to 16 MiB points over several fresh huge-page allocations (and with `hugetlbfs` pages) and check whether the LLC-miss rate varies from allocation to allocation.

In DRAM the random 4 KiB-page chase incurs 0.988 completed page walks and 1.99 L1D misses per access (versus 1.00 with 2 MiB pages): the page walker's own page-table reads miss the L1 too, which is why the 4 KiB curve sits above the 2 MiB curve (106 ns vs. 85.1 ns at 512 MiB). The sequential chase costs only 7.82 ns in DRAM with 0.0686 demand LLC misses per access versus 1.000 for random: the prefetcher turns its misses into hits.

Transitions are gradual rather than sharp. At 192 KiB (75% of L2) the 4 KiB-page chase already costs 6.68 ns with 0.197 L2 misses per access, while the 2 MiB-page chase stays at 4.36 ns (0.00248): almost no L2 misses. The likely explanation is page placement: 4 KiB pages scatter the buffer over arbitrary physical frames, so some L2 sets receive more lines than their associativity allows (conflict misses), whereas a physically contiguous 2 MiB page spreads lines evenly over the sets. The placement is decided per allocation, which explains why the 4 KiB L2 window is not monotonic: each size is a fresh buffer with its own set of frames (page colours), and at 96 KiB the frames happened to cover the sets evenly (4.33 ns, the L2 hit latency, against 6.18 ns at 64 KiB). Each size runs in its own process, whose 8 repetitions reuse one buffer and agree closely. A different allocation of the same size can differ even with the identical ring (same seed): the zero-queue run of Section 4 measured 4.31 ns at 64 KiB and 6.10 ns at 128 KiB, the reverse of this sweep's 6.18 ns and 5.22 ns.



(a) Runtime and fitted AMAT model.

(b) Counters per element.

Figure 7: SAXPY time per element and counters vs. footprint and traversal order (two `double` arrays, $16N$ bytes; lines mark where $16N$ crosses L1/L2/L3; random order also reads a $4N$-byte index array, so its $20N$-byte footprint crosses each capacity at $0.8\times$ the marked N). Open circles: AMAT model fitted to the counters (Equation (1)).

This staircase is the input to the AMAT model in Section 9. Counter multiplexing was in effect for some events (minimum running fraction 0.662); counts are scaled accordingly.

9 Cache-Miss Impact

Hypothesis (from first principles, then compared with the data)

As the SAXPY footprint grows past each cache, time per element should rise in proportion to the misses per element at each level. Sequential traversal should rise gently—the prefetcher turns misses into hits—while a random permutation should rise sharply once the set leaves cache. Because independent iterations overlap their misses, the effective cost of a DRAM miss should be well below the zero-queue DRAM latency.

Kernel	$y_i = a\,x_i + y_i$ (float64), two arrays, 4 KiB pages
Varied	$N = 2^8 \ldots 2^{24}$ (footprint $16N$ bytes sequential; $20N$ random, which adds a 4-byte index per element); order (sequential, random permutation)
Counters	L1D read misses, LLC read references/misses, dTLB walks, cycles, instructions (user space, per element)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh cache_miss`

9.1 Analysis

Sequential SAXPY rises from 0.151 ns per element in L1 to 0.918 ns in DRAM, a 6.1× span, with only 0.0549 demand LLC misses per element at the largest size: the prefetcher has already brought most lines in. Random traversal rises from 0.812 ns to 29.9 ns, a 37× span, with 3.98 L1D misses, 1.96 LLC misses and 1.70 page walks per element at $N = 2^{24}$ (each element touches a line of x, a

line of y, plus page-walk reads). At DRAM sizes the random curve is 33× the sequential one: that gap *is* the miss cost the prefetcher hides for regular code.

9.2 AMAT fitted to measured miss counts

Classic AMAT, $t_{L1} + m_{L1}(t_{L2} + m_{L2}(t_{L3} + m_{L3}\, t_{\mathrm{DRAM}}))$, assumes misses are serviced one at a time. We instead fit its per-element form to the counters, separately for each order:

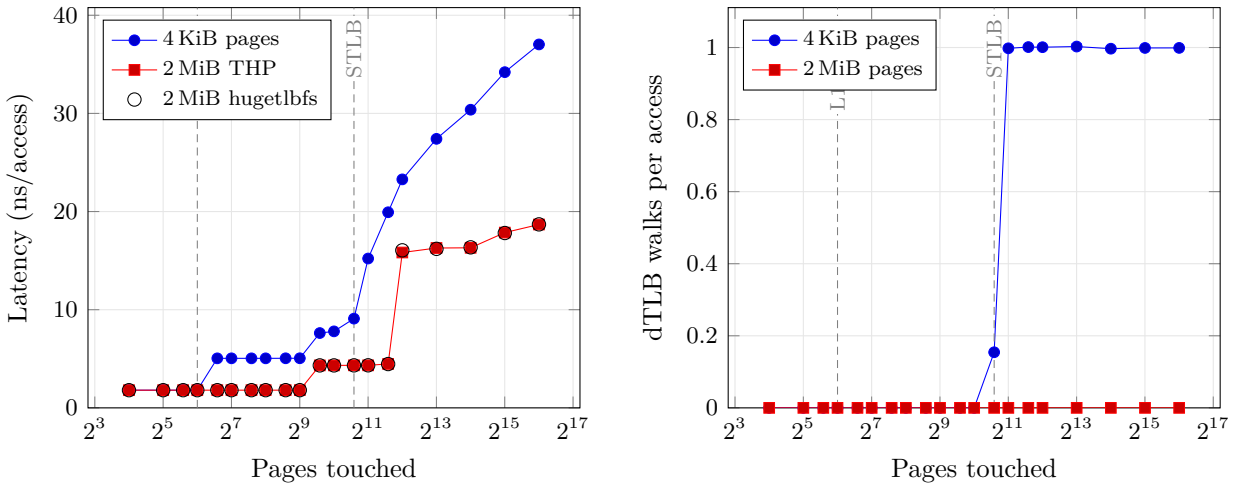
$$t_{\mathrm{elem}} = t_0 + a_2\, h_{L2} + a_3\, h_{L3} + a_D\, m_D, \tag{1}$$

where h_{L2} = L1D misses − LLC references (served by L2), h_{L3} = LLC references − LLC misses (served by L3) and m_D = LLC misses (served by DRAM), all per element; t_0 absorbs the L1-hit work. Ordinary least squares over the 17 footprints gives, for random order, $t_0 = 0.533$ ns, $a_2 = 0.502$ ns, $a_3 = 2.12$ ns, $a_D = 12.3$ ns ($R^2 = 0.986$); for sequential order $a_D = 8.37$ ns ($R^2 = 0.999$). The fitted a_D is the *exposed* cost of one DRAM miss. Dividing the zero-queue DRAM latency by it gives the average number of DRAM misses overlapped: 8.3 for random and 12 for sequential SAXPY. For random order the same reading holds at every level: the zero-queue L2 and L3 latencies are 10× and 8.8× the fitted a_2 and a_3. All three overlap factors (8.3–10) are close to the 8.70 demand misses one core keeps in flight in Section 7: independent SAXPY iterations fill the same fill buffers that the chase experiment measured. Classic AMAT with serialised misses makes the same point at $N = 2^{24}$: with the Table 2 latencies (L2 5.21 ns, L3 18.6 ns, DRAM 103 ns), the measured $h_{L2} = 0.33$, $h_{L3} = 1.68$ and $m_D = 1.96$ per element, and the fitted t_0, it predicts 236 ns per element (202 ns of it DRAM) against the measured 29.9 ns, a 7.9-fold overprediction, close to the 8.3-fold DRAM overlap factor. Sequential order overlaps more (12 for DRAM) because the misses that remain after prefetching arrive in long, predictable runs. Caveats: all coefficients came out positive, but the fit is a four-parameter regression on 17 footprints, so the individual L2/L3 terms are only roughly determined. The random-order fit has its largest relative errors (up to 34%; sequential: 6.6%) at two footprints, for different reasons. At the smallest, $N = 2^8$ (34%), one shared t_0 cannot describe the index-load and loop overhead. At $N = 2^{20}$ (33%: 10.3 ns measured, 6.84 ns predicted) the error comes from the L3→DRAM transition, not from t_0. There the random footprint, $20N$ bytes, equals the 20 MiB L3, so only 0.123 LLC misses occur per element, but the 4 KiB pages already need 1.03 page walks per element. The page-walk cost is folded into a_D because walks and DRAM misses grow together over most of the sweep, so the model has no term for walks without DRAM misses. With 2 MiB pages the same N runs at 5.42 ns per element (Table 4), which shows that the missing time is the walks (Section 10 separates them).

10 TLB-Miss Impact

Hypothesis (from first principles, then compared with the data)

With 4 KiB pages, a dependent access to a random page should cost extra once the number of pages touched exceeds the L1 dTLB (64 entries) and again past the second-level TLB (1536 entries, ≈6 MiB of reach), where every access needs a page walk. With 2 MiB pages one entry covers 512× more memory, so the same page counts should need no walks and the extra latency should vanish.



(a) Page-stride chase latency.

(b) Completed page walks (counter).

Figure 8: TLB reach: dependent access to one line per page. The gap between the curves is the cost of TLB misses; the counter confirms walks occur only with 4 KiB pages past the STLB reach.

Chase	one node per 4 KiB page (node stride 4160 B = page + one line, so nodes fall into different cache sets); random order; 16–65536 pages
SAXPY	random-order $y_i = a\,x_i + y_i$, $N = 2^{18} \ldots 2^{24}$
Varied	page size: 4 KiB (`MADV_NOHUGEPAGE`) vs. 2 MiB transparent huge pages (`MADV_HUGEPAGE`), both set before first touch; the chase also uses reserved `hugetlbfs` 2 MiB pages (`MAP_HUGETLB`)
Verification	`AnonHugePages` per run; dTLB-walk counter per access
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh tlb`

The page-stride chase is designed so that page size is the *only* difference between the two curves: both touch the same number of cache lines (one per page, so the cache footprint is N lines), and the one-line stagger avoids every node mapping to the same cache set. Any latency gap is therefore TLB reach and page-walk cost.

10.1 Analysis

The curves show both TLB levels as separate cliffs (Figure 8). Up to 64 pages both page sizes cost the same (1.80 ns vs. 1.80 ns). At 128 pages the 4 KiB chase jumps to 5.05 ns while the 2 MiB chase stays at 1.80 ns, with zero page walks in both. The 3.25 ns (9.10 cycles) gap is the cost of missing the 64-entry L1 dTLB and hitting the second-level TLB. Near the STLB capacity walks begin (0.154 per access at 1536 pages), and at 2048 pages every 4 KiB access walks (0.998 walks per access): 15.2 ns against 4.33 ns, a 10.9 ns (30.4-cycle) page-walk penalty while the page-table entries still hit in cache. Past 4096 pages the cache footprint itself leaves L2 (4096 lines = 256 KiB), raising both curves—the huge-page chase rises to 15.8 ns—so only the gap is attributed to the TLB. That gap grows again as the page tables outgrow the caches: 14.1 ns at 16384 pages and 18.3 ns at 65536 pages (37.0 ns vs. 18.7 ns, 1.98×). With 2 MiB pages the walk counter stays at 0 even at 65536 pages.

For random SAXPY (Table 4) at $N = 2^{24}$, 2 MiB pages cut page walks from 1.70 to 0 per element and runtime from 30.0 ns (this run; the separate `cache_miss` run of Section 9 measured 29.9 ns) to 16.9 ns per element, a 1.78× speedup, a little below the 1.98× of the dependent chase. Overlapping independent misses hides only a small part of the walks: each walk adds dependent

Table 4: Random-order SAXPY with 4 KiB vs. 2 MiB pages (`tlb` experiment; median of 8 repetitions). Footprint = $20N$ bytes (x, y and the 4-byte index array). t: ns per element; w: completed page walks per element; m: LLC misses per element; subscripts give the page size; speedup = t_{4K}/t_{2M}. Generated from `data/tlb_saxpy_table.csv` (derived from `data/tlb_saxpy.csv`).

N	MiB	t_{4K}	t_{2M}	w_{4K}	w_{2M}	m_{4K}	m_{2M}	speedup
2^{18}	5	3.83	3.79	0.00	0.00	0.00	0.00	1.01
2^{19}	10	4.64	3.86	0.39	0.00	0.00	0.00	1.20
2^{20}	20	10.3	5.42	1.03	0.00	0.12	0.04	1.89
2^{21}	40	15.8	10.6	1.36	0.00	0.98	0.96	1.48
2^{22}	80	21	14.2	1.54	0.00	1.55	1.54	1.48
2^{23}	160	26.5	16.1	1.65	0.00	1.83	1.81	1.65
2^{24}	320	30	16.9	1.70	0.00	1.96	1.94	1.77

page-table reads in front of its data miss. The largest speedup in the sweep is at $N = 2^{20}$, where the footprint equals the L3: the 2 MiB-page data still mostly hit in the L3, while the 4 KiB pages already need about one walk per element (Section 9).

Two huge-page mechanisms agree. Reserved `hugetlbfs` pages (`MAP_HUGETLB`, from the 1024 pages the user set aside) give the same chase latency as transparent huge pages at every size (16.0 ns vs. 15.8 ns at 4096 pages, 18.7 ns vs. 18.7 ns at 65536), with zero walks. A `MAP_HUGETLB` mapping either gets reserved pages or fails, so no grant check is needed (these pages are not counted in `AnonHugePages`). The THP result is therefore not an artifact of how the kernel assembled the pages.

Huge pages are verified, not assumed. Requesting huge pages does not guarantee them: a benchmark that only asks for THP can silently run on 4 KiB pages and report "no effect" (Section 12). The benchmarks map 2 MiB-aligned regions and call `madvise` before the first touch; each row records the granted `AnonHugePages` (summarised in the run's `anon_thp.txt`), `validate.sh` checks the grant before each experiment, and the dTLB-walk counter independently confirms the page size in effect. In this run every "huge" configuration was fully backed: `AnonHugePages` equals the buffer size at every point (for example 266 240 KiB for the 65536-page chase, and 327 680 KiB for SAXPY at $N = 2^{24}$, i.e. x, y and the index array), and every 4 KiB row records 0 KiB.

11 Benchmark Pitfall

Hypothesis (from first principles, then compared with the data)

A "DRAM bandwidth" benchmark that re-streams a buffer small enough to live in cache will report a number several times higher than the real memory bandwidth, because every pass after the first hits the cache instead of DRAM. We expect a buffer that fits in L1/L2 to report tens of GiB/s and a DRAM-resident buffer to settle near the single-core streaming figure from Section 5.

Pitfall	Measuring cache reuse instead of DRAM: picking a buffer that fits in cache
Wrong setup	Working set 16 KiB (16 320 B after rounding to whole loop units; L1-resident), re-streamed many times, one thread
Right setup	Working set 64 MiB to 512 MiB (well past the 20 MiB LLC)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh pitfall`

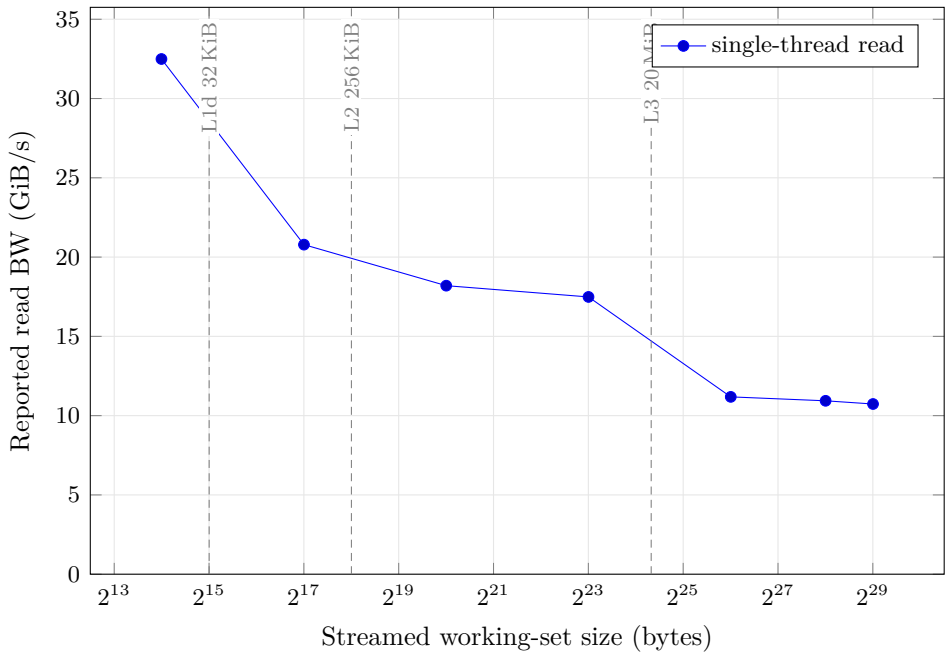


Figure 9: The same streaming kernel "measures" wildly different bandwidth depending only on the buffer size. A 16 KiB buffer reports 32.5 GiB/s; the honest DRAM-resident figure is 10.7–11.2 GiB/s. Nothing about the code changed—only where the data lived.

11.1 Explanation

At 16 KiB the kernel reports 32.5 GiB/s, falling as the set spills L1 and L2; once the set is 64 MiB or larger—genuinely DRAM-resident—it settles at 10.7–11.2 GiB/s, a 3.0× overstatement for the small buffer. (This kernel is the scalar unit-stride read of Section 5; its DRAM figure is window-limited and is not the single-core peak.) The inflated small-buffer number is not DRAM bandwidth at all: it is L1/L2 bandwidth, reported by a loop that re-reads the same cached lines. An AI-generated bandwidth microbenchmark very easily falls into this trap, because a 16 KiB array is a perfectly natural default and the resulting tens of GiB/s look like a plausible "memory" number. Two cheap detectors flag it: (1) the reported bandwidth *changes with buffer size*, which a true DRAM measurement must not once the set exceeds the LLC; and (2) the small-buffer figure from *one* core matches or exceeds what the whole memory system delivers to *all* cores (32.1 GiB/s, Section 7). Our methodology avoids the pitfall by sizing every bandwidth and latency point explicitly against the cache capacities (Section 2) and, for the DRAM claims, using working sets at least 3× the LLC so the first-touch pass cannot leave the data cached.

12 Anomalies and Limitations

12.1 Unexpected results and how they were resolved

- **An ACPI event storm stole wall time from every core.** In a first complete run (its benchmark data were discarded and not retained), the in-process add-chain frequency probe read either 2.79 GHz or about 2.41 GHz with turbo off, which would have understated DRAM latency by about 14% in cycles. The diagnostic `clock_check` run below is retained. Hypothesis: the clock does not change, but the thread loses wall time. Test: `clock_check`

compared core cycles, reference cycles and wall time in 80 windows of 330 ms on CPUs 2 and 6. The core clock was 2.79 GHz in every window. In 39% of the windows a burst of ACPI general-purpose events occurred (about 114 per window), and in those windows the pinned thread executed user code for only 0.866 of the wall time (quiet windows: 0.998), with no system time or context switches charged to it. The probe read 2.42 GHz in those windows (quiet: 2.79 GHz). The lost time is invisible to the OS and hits every core, consistent with firmware (SMM) handling of the events. The source, GPE 0x42, was then masked by the user, and all data in this report were re-measured. In the final `clock_check` run the thread keeps 0.997–0.998 of the wall time, and the probe reads 2.78–2.79 GHz in every single-thread run (Section 2). Cycles are still converted with the fixed 2.8 GHz clock, which does not depend on the probe (Section 2).

- **The add-chain probe is code-alignment sensitive.** A one-ADD loop ran at one or two cycles per iteration depending on where the compiler placed it, so some builds read half the clock. The probe now uses a 32-byte-aligned loop of eight dependent ADDs.
- **Unit-stride "peak" versus strided line traffic.** At 64 B stride the line traffic (15.7 GiB/s) exceeds the unit-stride scalar read (10.8 GiB/s), the kernel one would naturally use as the single-core peak. Hypothesis: the scalar kernel is limited by the core's out-of-order window, not by DRAM. Test: large full-line vector blocks reach 14.9 GiB/s on the same data, so the single-core limit is 15 GiB/s to 16 GiB/s (Section 5).
- $L/N = 1$ **is not a test.** In the known-concurrency sweep, λ times the per-chain step time equals N by construction. The informative quantity is L_{mem} with the probe's memory latency, which saturates at 8.70 lines per core (Section 7).
- **A single-core chase outlier.** With one generator core, $k = 12$ chains deliver less bandwidth than $k = 10$ or $k = 16$ at the same memory latency. The probe is unaffected, so the cause is in the generator loop, not in memory. Unresolved; only the envelope is used. (In the run disturbed by the ACPI storm, $k = 8$ and $k = 20$ were also low; data not retained, so this cannot be checked from the repository.)
- **The two unloaded DRAM latencies agree once the interference is gone.** In the disturbed run (data not retained), the `mem_loaded` probe and `mem_latency` differed by 6% at 256 MiB. In the final run they give 103 ns and 103 ns, consistent with the gap having been run-to-run noise from the stolen wall time rather than a property of the two rings.
- **Element-level read/write mixes measure the wrong thing.** Mixing 8 B elements reads every source line and writes every destination line, so a nominal 70/30 mix moves about three lines per useful line; mixes must be defined on lines (Section 6).
- **Requested huge pages are not necessarily granted—and vice versa.** An early TLB run (data not retained) requested THP and received `AnonHugePages:  0 kB`, so its "huge" curve was really 4 KiB pages. Conversely, with THP set to always a 2 MiB-aligned "4 KiB" buffer is backed by huge pages unless the program opts out. Both page policies are therefore set explicitly before first touch and verified per run (Section 10).

12.2 Limitations

- **DRAM counters are system-wide.** The memory-controller counts include all traffic in the machine; an idle window carries 0.088 GiB/s of reads and 0.025 GiB/s of writes (`clock_check`),

which is small next to every loaded measurement but is not subtracted. Core events are counted in user mode for the measuring thread only; generic LLC events map to offcore-response events on this core and may undercount prefetch-driven fills. Core counters are multiplexed when more events are requested than free hardware counters; counts are scaled and the running fraction is recorded.

- **Single-core traffic model not closed.** With one generator core the memory-controller counters read 0.945–0.962× the RFO-counted traffic estimate (Section 6); the cause is an untested hypothesis.
- **MLC ran with prefetchers enabled.** Without root, MLC cannot switch the hardware prefetchers off, so its latencies use random access (`-r`) with prefetchers on, like ours.
- **Loaded latency is measured with a probe that is itself load.** The probe adds one line in flight (included in throughput) and measures a demand read; the streaming generators' prefetches may see a different latency, so $L = \lambda W$ for the streaming sweep assumes the probe's latency is representative. The known-concurrency chase sweep does not need that assumption.
- **One machine and one memory configuration.** All numbers come from a single i9-10900F on one NUMA node; DIMM population and speed could not be read without root, so no theoretical DRAM peak is quoted (Section 7). Frequency policy for the runs: governor performance, turbo disabled, SMT off.
- **Report build backend.** The host has `bibtex` but not `biber`, so `biblatex` uses `backend=bibtex`.

13 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used

Claude Code throughout, as recorded in `AI_USE_LOG.md`. With Claude Opus 5: on 2026-09-14 it produced an unrequested full implementation, which was rejected and deleted the next day; on 2026-09-15 it produced the project scaffold (templates, Makefiles, report skeletons), which was kept. The first Project 2 benchmarks and report followed on 2026-10-20, after a build check of the scaffold on 2026-10-17. With Claude Opus 5.5, the 2026-10-21 rework produced every measurement in this report. The builds used gcc (GCC) 16.2.1 20260810. The AI wrote the code and the prose. Claude Code also ran the timed experiments, on the user's machine (Table 1) and with the user's approval, after the user had made the root-only settings: performance governor, turbo off, SMT siblings offline, the ACPI event source masked, `perf_event_paranoid`=0, 1024 reserved huge pages, and Intel MLC installed. Every number comes from those runs.

What they helped generate

The four microbenchmarks—`mem_latency` (Sattolo pointer chase with access-size, page-size and counter options), `mem_bandwidth` (multi-thread streaming with read/write/copy/triad/rwmix and access-size modes), `mem_loaded` (loaded-latency probe with streaming, read/write and known-concurrency generators) and `kernel_miss` (footprint-varying SAXPY with counters)—plus the shared helpers, the `perf_event_open` counter code, the experiment driver (`run.sh`), `validate.sh`, `derive.py` and drafts of every section.

What was independently modified, tested, or rejected

Each item below is an AI output that was wrong and was caught by a check, a review or the data:

(1) The first latency benchmark used a *sequential* ring, so the prefetcher turned it into a ∼5 ns "latency". It was replaced by a random Sattolo cycle; the sequential curve is kept only as a contrast (Figure 6).

(2) The first huge-page experiment assumed THP backing because THP was set to `always`. A runtime `smaps_rollup` check showed `AnonHugePages: 0 kB`, so its "huge pages don't help" would have been false. The allocation was rebuilt (2 MiB-aligned mapping, `madvise` before first touch); the grant is now verified for every run, and huge pages give a 1.78× speedup (Section 10).

(3) The converse also happened: under THP `always`, a "4 KiB" buffer could silently get huge pages. `MADV_NOHUGEPAGE` is now set explicitly.

(4) An AI-suggested bandwidth test defaulted to a cache-resident buffer and reported tens of GiB/s as "DRAM bandwidth"; this is now the documented pitfall (Section 11).

(5) The read/write-mix kernel mixed 8 B elements, so every line moved in both directions; mixes are now line-granular (Section 6).

(6) The throughput–latency knee was placed by eye, and Little's Law used the zero-queue latency. Both were replaced by a concurrent loaded-latency probe and a stated knee rule that follows offered load (Section 7).

(7) A "validation" of Little's Law ($L/N \approx 1$) turned out to be an accounting identity. It was replaced by L_{mem} with the probe-measured memory latency.

(8) The scalar unit-stride kernel was quoted as the single-core peak although the strided data contradicted it (Section 5).

(9) A hand-copied table disagreed with its CSV. All tables and quoted numbers are now generated from the data (`derive.py`, `summary.csv`).

(10) Cycle counts were first converted with the in-process add-chain frequency probe, which misread the clock in about a third of runs. An independent AI regrade session (Claude Code), run at the student's request, flagged this, and the AI's diagnostic traced it to wall time lost to an ACPI event storm, which the user then masked (Section 12). Conversion now uses the fixed 2.8 GHz clock, confirmed with the cycle counter.

In a final round, the AI added Intel MLC runs (`run.sh mlc`) as the external cross-check and system-wide memory-controller counters (`uncore_imc`) that measure DRAM read/write traffic. Both became possible after the user installed MLC and lowered `perf_event_paranoid`. All experiments were then re-run.

Validation

`scripts/validate.sh` asserts the physically necessary orderings (L1 < L2 < L3 < DRAM latency; L1 bandwidth > DRAM; multi-thread > single-thread; loaded > unloaded latency) and agreement of the two latency methods; its output is saved with every experiment. Hardware counters confirm the miss level behind every latency step. Intel MLC provides the external cross-check (Section 3), and the memory-controller counters check the traffic models (Sections 5 and 6). Every figure traces to a committed raw/processed file (Section A).

14 Conclusion

The hierarchy behaves as its capacities predict, and the counters show *why* each step occurs. Zero-queue latency is 1.80 ns (5.03 cycles) in L1, 5.21 ns on the L2 plateau with 4 KiB pages (L2 hit latency 4.33 ns without conflict misses), 18.6 ns in L3 and 103 ns in DRAM, of which 19.6 ns is page walking that 2 MiB pages remove. Concurrency is the other half of the story. Measured under load, bandwidth reaches 30.6 GiB/s at only 1.37× the unloaded latency, and past that point more cores buy queueing, not bandwidth; Intel MLC's sweep draws the same curve. With measured latency, Little's Law separates the 55.6 lines the memory system serves at saturation from up to 152 lines that merely queue; one core sustains at most 8.70 random demand misses, but far more lines when the prefetcher streams; and a counter-fitted AMAT model shows random SAXPY paying 12.3 ns per DRAM miss—the DRAM latency divided by 8.3-way overlap. Memory-controller counters turn two estimates into measurements: every written line costs one extra read (the RFO; within 1% at full load, a few percent below the model with one core), and sparse strided reads cost two lines each because of the adjacent-line prefetcher. The lessons are about measurement discipline as much as hardware: a bandwidth benchmark must out-size the LLC or it measures cache reuse (Section 11); a latency benchmark must randomize or it measures the prefetcher; a knee must come from measured latency, not throughput alone; a read/write mix must be defined at line granularity; a huge-page study must verify its pages—in both directions (Section 10); and a clock must be measured, not assumed, because firmware can silently steal wall time from every core (Section 12).

References

[1] Intel Corporation, *Intel memory latency checker*, Version 3.13 on the test host; used as an independent cross-check (random-access mode, prefetchers left enabled).

[2] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. DOI: 10.1287/opre.9.3.383

[3] *Perf: Linux profiling with performance counters*. [Online]. Available: https://perf.wiki.kernel.org/

A Reproducibility

Code, scripts, raw and processed data, and this report live in one repository with two top-level directories, `ACS-Code/` and `ACS-Latex/`, at commit `f5bfe7005a41`.

```
gh repo clone SKempiak/ACS-Code ACS && cd ACS
git checkout f5bfe7005a41
cd ACS-Code
make tools                                          # tools/summarize
make -C Cache_and_Memory_Performance_Profiling      # mem_latency mem_bandwidth mem_loaded
    kernel_miss
make -C Cache_and_Memory_Performance_Profiling test   # physical-ordering sanity checks (validate.
    sh)
scripts/collect_env.sh --tex > ../ACS-Latex/common/system_info.tex
Cache_and_Memory_Performance_Profiling/scripts/run.sh zero_queue pattern rw_mix intensity \
    working_set cache_miss tlb pitfall mlc clock_check   # or: run.sh all (CPU_PIN=2 by default)
scripts/sync_data.sh Cache_and_Memory_Performance_Profiling
cd ../ACS-Latex && make memory
```

Table 5: Raw data behind each figure and table (directories under ACS-Code/Cache_and_Memory_Performance_Profiling/data/raw/).

Figure / table	run.sh experiment	Processed CSV	Raw run directory
Table 2	zero_queue	zero_queue, zero_queue_table (L2 2 MiB value: working_set)	zero_queue/20261021-202254
Figures 1 and 2	pattern	pattern_lat, pattern_bw	pattern/20261021-202254
Figure 3	rw_mix	rw_mix, rw_mix_ref	rw_mix/20261021-202254
Figures 4 and 5, Table 3	intensity	intensity_loaded, intensity_mlp, littles_law, knee	intensity/20261021-202254
Figure 6	working_set	working_set	working_set/20261021-202254
Figure 7	cache_miss	cache_miss_ctr, amat_fit	cache_miss/20261021-202254
Figure 8	tlb	tlb_chase	tlb/20261021-202254
Table 4	tlb	tlb_saxpy, tlb_saxpy_table	tlb/20261021-202254
Figure 9	pitfall	pitfall	pitfall/20261021-202254
Table 2 and Figure 4, Section 6 (MLC)	mlc	mlc_idle, mlc_loaded, mlc_bw	mlc/20261021-202254
Section 2 (clock, idle DRAM traffic)	clock_check	— (summary.csv)	clock_check/20261021-202254
Section 12 (ACPI storm)	clock_check	— (summary.csv)	clock_check/20261021-194648

Each experiment directory holds the raw rows (one per repetition, with full configuration metadata), env.txt, build.txt, commands.txt and validate.txt (Section 2), except that the MLC directory has no build.txt or validate.txt and the earlier clock_check run has no validate.txt. The processed CSVs are produced from them by tools/summarize; scripts/derive.py computes the derived tables and summary.csv, from which every number in the text is typeset; and scripts/sync_data.sh copies all of them into this report's data/. The measured code is exactly commit b00db66: the runs started from an uncommitted tree whose sources, scripts and Makefile match that commit byte for byte (Section 2). All directories except the last come from a single run.sh all invocation (2026-10-21, 20:22–20:39; one shared timestamp). The earlier clock_check run (clock_check/20261021-194648), taken before the ACPI event source was masked, is kept as the evidence for Section 12. MLC's raw text output is in its directory next to the parsed CSVs (scripts/mlc_to_csv.py).